EscuelaMAK — Documento Técnico Completo

> Versión 1.0 | Plataforma Educativa Móvil | Flutter + Supabase

---

1. RECOMENDACIÓN TECNOLÓGICA

¿Por qué Flutter + Dart?

Criterio	Flutter	React Native
Curva de aprendizaje	✅ Más suave para principiantes	⚠️ Requiere JS previo
Documentación en español	✅ Extensa y oficial	⚠️ Parcial
Rendimiento	✅ Compilado nativo	⚠️ Puente JS
Hot Reload	✅ Instantáneo	✅ Disponible
Material Design 3	✅ Nativo incluido	⚠️ Librería externa
Una sola base de código	✅ Android + iOS + Web	✅ Android + iOS
Respaldo corporativo	✅ Google	✅ Meta

Veredicto: Flutter es la mejor opción para este proyecto. Su sistema de widgets reduce el código en un 40% comparado con otras alternativas, y su integración con Supabase es directa y bien documentada.

---

2. ARQUITECTURA DE LA APLICACIÓN

2.1 Patrón: Clean Architecture

```
┌──────────────────────────────────────┐
│          PRESENTATION LAYER          │
│   Screens · Widgets · Providers      │
├──────────────────────────────────────┤
│            DOMAIN LAYER              │
│   UseCases · Entities · Interfaces   │
├──────────────────────────────────────┤
│             DATA LAYER               │
│  Repositories · DataSources · DTOs   │
├──────────────────────────────────────┤
```

```
│          EXTERNAL SERVICES           │
│    Supabase · Storage · Auth · Push  │
└──────────────────────────────────────┘
```

2.2 Flujo de Datos

```
Usuario → Screen → Provider (Estado) → UseCase → Repository → Supabase
                ↑                                                ↓
              UI Update ←←←←←←← Response/Stream ←←←←←←←←←←←
```

2.3 Patrones de Diseño Utilizados

- **Repository Pattern:** Abstrae el acceso a datos de Supabase
- **Provider Pattern:** Manejo de estado reactivo con Riverpod
- **Observer Pattern:** Streams de Supabase para tiempo real
- **Factory Pattern:** Creación de modelos desde JSON
- **Singleton Pattern:** Cliente Supabase global

---

3. ESTRUCTURA DE CARPETAS

```
escuelamak/
├── lib/
│   ├── main.dart                          # Punto de entrada
│   ├── app.dart                           # Configuración raíz
│   │
│   ├── core/                              # Núcleo compartido
│   │   ├── constants/
│   │   │   ├── app_colors.dart
│   │   │   ├── app_strings.dart
│   │   │   └── supabase_keys.dart
│   │   ├── errors/
│   │   │   ├── app_exception.dart
│   │   │   └── failure.dart
│   │   ├── router/
│   │   │   └── app_router.dart
│   │   ├── theme/
│   │   │   └── app_theme.dart
│   │   └── utils/
│   │       ├── date_utils.dart
│   │       └── validators.dart
│   │
│   ├── features/                          # Módulos por funcionalidad
```

```
│   │   ├── auth/
│   │   │   ├── data/
│   │   │   │   ├── auth_repository.dart
│   │   │   │   └── auth_datasource.dart
│   │   │   ├── domain/
│   │   │   │   ├── auth_usecase.dart
│   │   │   │   └── user_entity.dart
│   │   │   └── presentation/
│   │   │       ├── login_screen.dart
│   │   │       ├── register_screen.dart
│   │   │       └── auth_provider.dart
│   │   │
│   │   ├── courses/
│   │   │   ├── data/
│   │   │   ├── domain/
│   │   │   └── presentation/
│   │   │
│   │   ├── tasks/
│   │   ├── quizzes/
│   │   ├── messages/
│   │   ├── calendar/
│   │   ├── permissions/
│   │   └── stats/
│   │
│   └── shared/                           # Widgets reutilizables
│       ├── widgets/
│       │   ├── custom_button.dart
│       │   ├── custom_text_field.dart
│       │   ├── loading_widget.dart
│       │   └── error_widget.dart
│       └── models/
│           └── pagination_model.dart
│
├── assets/
│   ├── images/
│   ├── icons/
│   └── fonts/
│
├── test/
│   ├── unit/
│   ├── widget/
│   └── integration/
│
├── pubspec.yaml
└── README.md
```

4. DEPENDENCIAS — pubspec.yaml

```yaml
```

```yaml
name: escuelamak
description: Plataforma educativa móvil EscuelaMAK
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter

  # Backend y autenticación
  supabase_flutter: ^2.3.0      # Cliente oficial Supabase

  # Manejo de estado
  flutter_riverpod: ^2.4.9      # Gestión de estado reactiva
  riverpod_annotation: ^2.3.3   # Generación de código Riverpod

  # Navegación
  go_router: ^13.2.0            # Navegación declarativa

  # UI y diseño
  google_fonts: ^6.1.0          # Tipografías profesionales
  flutter_svg: ^2.0.9           # Iconos SVG
  cached_network_image: ^3.3.1  # Caché de imágenes de red
  shimmer: ^3.0.0               # Efecto loading skeleton
  fl_chart: ^0.66.2             # Gráficos y estadísticas
  table_calendar: ^3.0.9        # Calendario de actividades

  # Multimedia
  video_player: ^2.8.2          # Reproducción de videos
  chewie: ^1.7.4                # Controles de video profesionales
  image_picker: ^1.0.7          # Selección de imágenes/archivos
  file_picker: ^6.1.1           # Selección de archivos genéricos

  # Utilidades
  intl: ^0.19.0                 # Fechas y localización
  shared_preferences: ^2.2.2    # Almacenamiento local simple
  flutter_secure_storage: ^9.0.0 # Almacenamiento seguro de tokens
  connectivity_plus: ^5.0.2     # Detección de conexión a internet
  permission_handler: ^11.1.0   # Permisos del dispositivo
  uuid: ^4.3.3                  # Generación de UUIDs
  timeago: ^3.6.0               # Fechas relativas ("hace 5 min")

  # Notificaciones
  firebase_messaging: ^14.7.18  # Push notifications
```

```
  flutter_local_notifications: ^16.3.0 # Notificaciones locales

  # Formularios y validación
  reactive_forms: ^16.1.1       # Formularios reactivos

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0
  build_runner: ^2.4.8
  riverpod_generator: ^2.3.9
  json_serializable: ^6.7.1
  mockito: ^5.4.4
```

---

5. MÓDULOS — Orden de Implementación para Principiantes

Fase 1 — Fundamentos (Semanas 1-2)

1. ✅ Configurar Flutter + proyecto base
2. ✅ Conectar Supabase (ya tienes la base)
3. ✅ Implementar autenticación (login/registro)
4. ✅ Crear sistema de roles y navegación

Fase 2 — Contenido (Semanas 3-4)

5. 📚 Módulo de Cursos (listado + detalle)
6. 🎬 Reproductor de lecciones con video
7. 📎 Sistema de archivos con Supabase Storage

Fase 3 — Interactividad (Semanas 5-6)

8. 📝 Módulo de Tareas (crear/entregar)
9. ❓ Sistema de Quizzes con temporizador
10. 💬 Mensajería en tiempo real

Fase 4 — Gestión (Semanas 7-8)

11. 📅 Calendario de actividades
12. 🔐 Panel de permisos (admin)
13. 📊 Estadísticas y reportes

Fase 5 — Pulido y Despliegue (Semanas 9-10)

14. 🧪 Testing unitario y de widgets
15. 🚀 Build y publicación en stores

---

6. ROLES Y PERMISOS

Mapa de acceso por rol

Módulo	Estudiante	Profesor	Admin
Ver cursos	✅	✅	✅
Crear cursos	❌	✅	✅
Entregar tareas	✅	❌	✅
Calificar tareas	❌	✅	✅
Tomar quizzes	✅	❌	✅
Crear quizzes	❌	✅	✅
Mensajería	✅	✅	✅
Panel de admin	❌	❌	✅
Estadísticas	❌	Parcial	✅
Gestionar permisos	❌	❌	✅

---

7. HERRAMIENTAS RECOMENDADAS

IDE y Extensiones

- **VS Code** (recomendado para principiantes) + extensiones:
  - Flutter (oficial)
  - Dart (oficial)
  - Flutter Riverpod Snippets
  - Error Lens (muestra errores inline)
  - GitLens (control de versiones visual)

- Thunder Client (prueba APIs)

Servicios Externos

- **Supabase:** Base de datos + Auth + Storage *(ya configurado)*
- **Firebase:** Solo para Push Notifications
- **GitHub:** Control de versiones gratuito
- **Figma:** Diseño de pantallas (gratis para principiantes)
- **Postman:** Prueba de endpoints REST

Depuración y Testing

- Flutter DevTools (incluido con Flutter)
- Supabase Dashboard (logs en tiempo real)
- Android Studio Emulator o dispositivo físico

---

8. CONVENCIONES DE CÓDIGO

Nomenclatura Dart

```dart
// Clases: PascalCase
class AuthRepository {}

// Variables y funciones: camelCase
String userName = 'Juan';
void fetchCourses() {}

// Constantes: lowerCamelCase con 'k' prefix
const kPrimaryColor = Color(0xFF6750A4);

// Archivos: snake_case
// auth_repository.dart
// login_screen.dart
```

Estructura de Commits (Conventional Commits)

```
feat: agregar pantalla de login
fix: corregir error en subida de archivos
docs: actualizar README con instrucciones
style: aplicar formato a auth_screen.dart
```

```
refactor: simplificar lógica de permisos
test: agregar tests para AuthRepository
chore: actualizar dependencias pubspec.yaml
```

Manejo de Errores

```dart
// ✅ CORRECTO — siempre manejar errores
try {
  final result = await authRepository.login(email, password);
  result.fold(
    (failure) => showErrorSnackbar(failure.message),
    (user) => navigateToHome(user),
  );
} on AuthException catch (e) {
  showErrorSnackbar('Error de autenticación: ${e.message}');
} catch (e) {
  showErrorSnackbar('Error inesperado. Intenta de nuevo.');
}

// ❌ INCORRECTO — nunca dejar errores sin manejar
final result = await authRepository.login(email, password);
```

---

9. ESTRATEGIA DE TESTING

Tests Unitarios (lib → test/unit/)

```dart
```

```dart
// test/unit/auth_usecase_test.dart
void main() {
  group('AuthUseCase', () {
    test('login exitoso retorna User', () async {
      final mockRepo = MockAuthRepository();
      when(mockRepo.login(any, any))
          .thenAnswer((_) async => Right(testUser));

      final usecase = AuthUseCase(mockRepo);
      final result = await usecase.login('test@test.com', '123456');

      expect(result.isRight(), true);
    });
  });
}
```

Tests de Widgets

```dart
// test/widget/login_screen_test.dart
void main() {
  testWidgets('LoginScreen muestra campos email y password', (tester) async {
    await tester.pumpWidget(
      ProviderScope(child: MaterialApp(home: LoginScreen())),
    );
    expect(find.byKey(Key('email_field')), findsOneWidget);
    expect(find.byKey(Key('password_field')), findsOneWidget);
    expect(find.text('Iniciar Sesión'), findsOneWidget);
  });
}
```

---

10. GUÍA DE DESPLIEGUE

Android (APK / Google Play)

```bash
```

```
# 1. Generar keystore
keytool -genkey -v -keystore ~/escuelamak.keystore \
  -alias escuelamak -keyalg RSA -keysize 2048 -validity 10000

# 2. Build APK de release
flutter build apk --release

# 3. Build App Bundle para Play Store
flutter build appbundle --release

# Resultado en: build/app/outputs/bundle/release/app-release.aab
```

iOS (App Store)

```bash
# 1. Requiere Mac con Xcode instalado
# 2. Configurar certificados en Apple Developer ($99/año)

# 3. Build IPA
flutter build ios --release

# 4. Abrir Xcode y archivar
open ios/Runner.xcworkspace
# Product → Archive → Distribute App
```

Checklist antes de publicar

- [ ] Cambiar URL de Supabase a producción
- [ ] Deshabilitar logs de debug
- [ ] Configurar iconos en todos los tamaños
- [ ] Configurar splash screen
- [ ] Probar en dispositivo físico (no solo emulador)
- [ ] Revisar permisos en AndroidManifest.xml e Info.plist
- [ ] Configurar privacidad y términos de uso

---

11. APRENDIZAJE PROGRESIVO — Recursos Recomendados

Semana 1-2: Fundamentos Dart

- 🎥 Flutter Official Docs — Tutorial "Get Started"
- 🎥 Curso: "Dart desde cero" en YouTube (Fernando Herrera)
- 📖 Practica: crear widgets simples, Column/Row/ListView

Semana 3-4: Flutter Intermedio

- 🎥 Riverpod 2.0 — documentación oficial (riverpod.dev)
- 🎥 GoRouter — navegación con roles
- 📖 Practica: conectar la pantalla de login a Supabase real

Semana 5+: Funcionalidades Avanzadas

- 🎥 Supabase Flutter — canal oficial YouTube
- 🎥 Clean Architecture con Flutter — Reso Coder (YouTube)
- 📖 Practica: implementar un módulo completo por semana

Documento generado para EscuelaMAK v1.0 — Actualizar conforme avance el proyecto